

Architecting Real-Time Voice Command Systems on Windows: A Comprehensive Technical Analysis of Sub-100ms Intent Classification, Hybrid NLU Pipelines, and Synthetic Data Distillation

1. The Latency Imperative in Local Voice Interfaces

The paradigm shift toward local, privacy-centric computing has catalyzed the development of applications like WhisperLocal, which leverages the formidable capabilities of OpenAI's Whisper architecture for Automatic Speech Recognition (ASR). However, transcribing speech is merely the perceptual layer of a voice interface; the cognitive layer—determining user intent—remains a significant bottleneck. Current implementations that couple real-time transcription with localized Large Language Models (LLMs) such as Llama 3 (8B parameters) introduce a latency penalty ranging from 2 to 12 seconds per interaction. In the context of Human-Computer Interaction (HCI), this delay is catastrophic. It fractures the cognitive feedback loop essential for seamless dictation and command execution, transforming what should be a fluid interaction into a disjointed, high-friction experience.

This report provides an exhaustive architectural blueprint for eliminating this bottleneck. It posits that the reliance on generative LLMs for the discriminative task of intent classification is fundamentally inefficient. By transitioning to a hybrid, cascading architecture rooted in highly optimized Small Language Models (SLMs) and distilled Transformer encoders, it is possible to achieve sub-100ms inference times on consumer Windows hardware without sacrificing the semantic understanding required for complex command disambiguation.

1.1 The Psychophysics of Response Time

To understand the engineering constraints of a voice dictation system, one must first quantify the user's perception of time. The seminal research by Miller (1968) and subsequent HCI studies establish three critical thresholds for system responsiveness:

1. **0.1 second (100ms):** The limit for having the user feel that the system is reacting instantaneously. This is the "tactile" threshold. In voice interfaces, this corresponds to the system acknowledging a command (e.g., visual feedback) or executing a formatting instruction immediately after the utterance concludes.
2. **1.0 second:** The limit for the user's flow of thought to stay uninterrupted, even though they notice the delay. For a dictation user, waiting more than a second to see if a command like "new paragraph" was understood breaks their creative momentum.

3. **10 seconds:** The limit for keeping the user's attention focused on the dialogue. The current 2-12 second latency of local LLMs places WhisperLocal squarely in this danger zone, where users are likely to disengage or double-check the system, assuming failure.

For a tool designed to replace the keyboard, latency is not merely a performance metric; it is the primary determinant of utility. A keyboard provides feedback in milliseconds. A voice interface attempting to emulate keyboard commands (like "delete word") must compete with this benchmark. Therefore, the architectural target must be sub-100ms for the vast majority (P99) of interactions.

1.2 The Computational Asymmetry: Generative vs. Discriminative Inference

The core inefficiency in the current architecture stems from using a generative model for a classification task. LLMs like Llama 3 are autoregressive; they generate output token by token, a process inherently bound by memory bandwidth. To classify an intent, the model must load gigabytes of weights into VRAM, process the input tokens, and then sequentially generate the classification label (e.g., "Intent: Delete"). Even with aggressive quantization (e.g., 4-bit GGUF), the sheer volume of memory operations required to move 8 billion parameters through the compute units creates an irreducible latency floor on consumer hardware.¹

In stark contrast, discriminative models such as BERT (Bidirectional Encoder Representations from Transformers) and its distilled variants (TinyBERT, MobileBERT) operate as pure encoders. They process the entire input sequence in a single forward pass, producing a dense vector representation that is immediately mapped to a class probability via a lightweight classification head. There is no generation loop, no KV-cache management, and the parameter count is often two to three orders of magnitude smaller. For example, TinyBERT-4 (14.5M parameters) is approximately 550 times smaller than Llama 3 8B. This reduction enables it to reside entirely within the CPU's L3 cache or a tiny fraction of GPU VRAM, allowing for inference speeds that are limited only by compute throughput rather than memory bandwidth.³

1.3 The Windows Hardware Landscape: A Heterogeneous Challenge

Developing for "offline Windows" introduces a unique set of constraints compared to standardized server environments. The user base will possess a wide spectrum of hardware capabilities:

- **The High-End Workstation:** Equipped with NVIDIA RTX 40-series GPUs, capable of running unquantized models with ease.
- **The Enterprise Laptop:** Running on Intel Core Ultra or AMD Ryzen AI processors, potentially lacking discrete GPUs but featuring Neural Processing Units (NPUs) and robust AVX-512 support.
- **The Legacy Machine:** Older desktops with integrated graphics and limited RAM.

A uniform solution relying on CUDA is insufficient, as it alienates users with AMD or Intel graphics. The architecture must leverage the **Open Neural Network Exchange (ONNX Runtime)**, which provides a unified API for hardware acceleration. On Windows, this involves utilizing the **DirectML Execution Provider**, which abstracts the GPU hardware via DirectX 12, allowing models to run efficiently on any modern GPU. Furthermore, for the smallest models, the CPU Execution Provider (utilizing MLAS and MKLDNN) often outperforms the GPU due to the elimination of PCIe data transfer overhead.⁵

2. Model Selection for Sub-100ms Inference

To achieve the sub-100ms target, we must rigorously evaluate model architectures that balance semantic understanding with raw inference speed. The leading candidates fall into two categories: distilled Transformers and sentence-embedding models.

2.1 Distilled Transformer Architectures

Knowledge Distillation (KD) is the primary technique for compressing the reasoning capabilities of large models into compact architectures suitable for real-time inference. In this process, a small "student" model is trained to reproduce the behavior (logits and hidden states) of a larger "teacher" model.³

2.1.1 TinyBERT: The Latency Champion

TinyBERT represents the state-of-the-art in transformer compression. It employs a two-stage distillation framework: general distillation (pre-training) and task-specific distillation (fine-tuning).

- **Architecture:** TinyBERT-4 consists of 4 transformer layers, a hidden size of 312, and 12 attention heads, totaling ~14.5 million parameters.³
- **Performance:** Empirical benchmarks demonstrate that TinyBERT-4 retains approximately 96.8% of BERT-Base's performance on the GLUE benchmark while being 7.5x smaller and 9.4x faster.
- **Latency Analysis:** On a standard Intel CPU, TinyBERT-4 inference latency for short sequences (typical of voice commands) is consistently in the **2-5 millisecond** range.¹¹ This creates a massive budget for pre- and post-processing logic within the 100ms window.
- **Suitability:** Its compact size allows it to be permanently loaded into memory with negligible footprint (~50MB), making it the ideal "always-on" classifier for the primary command loop.

2.1.2 MobileBERT: The Structural Optimizer

MobileBERT takes a different approach to compression. Rather than simply reducing depth or

width, it employs a bottleneck structure that allows it to maintain the depth of BERT-Large (24 layers) while drastically reducing the number of parameters via linear projections.¹

- **Architecture:** 24 layers, but with "thin" bottleneck layers that reduce computational complexity. Total parameters: ~25.3 million.
- **Performance:** MobileBERT achieves a GLUE score of 77.7, slightly underperforming BERT-Base but offering significant speedups.
- **Latency Analysis:** While faster than BERT, it is generally slower than TinyBERT due to its depth. On mobile CPUs (Pixel 4), it clocks ~62ms. On desktop CPUs, this translates to roughly **15-25ms**.
- **Suitability:** MobileBERT is a strong candidate if TinyBERT proves insufficient for capturing complex syntactic nuances, but for simple command recognition, it may be over-engineered.

2.1.3 ModernBERT: The Next-Generation Encoder

ModernBERT¹³ integrates recent architectural advances from the LLM era into the encoder-only paradigm.

- **Innovations:** It utilizes Rotary Positional Embeddings (RoPE), Flash Attention v2, and unpadding (removing padding tokens from calculations) to maximize throughput.
- **Context:** It supports sequence lengths up to 8192 tokens, which is largely irrelevant for short voice commands but beneficial for context-aware document editing.
- **Latency Analysis:** While extremely efficient for its size (149M parameters), the raw parameter count imposes a higher baseline latency than TinyBERT. On a CPU, inference may range from **40-80ms**. However, on a GPU with Flash Attention support, it can achieve extremely high throughput.
- **Suitability:** ModernBERT is likely too heavy for the primary CPU-based fast path but serves as an excellent intermediate fallback before escalating to the full LLM.

2.2 Embedding-Based Classifiers: The SetFit Approach

SetFit (Sentence Transformer Fine-tuning) offers a distinct paradigm. Instead of a classification head (softmax) atop the token embeddings, SetFit fine-tunes a Sentence Transformer to generate dense vector representations where semantically similar utterances cluster together.¹³

- **Mechanism:** The model encodes the input utterance into a vector. This vector is then compared (via cosine similarity) against a set of pre-computed "prototype" vectors representing each intent.
- **Few-Shot Learning:** SetFit excels in scenarios with limited training data, often achieving high accuracy with as few as 8 examples per class.
- **Latency:** The inference cost is dominated by the embedding generation. Using a lightweight backbone like all-MiniLM-L6-v2 (22M parameters), embedding takes **~10-15ms** on CPU. The similarity search is an $O(N)$ operation (where N is the number

of intents) and takes microseconds.

- **Dynamic Flexibility:** A critical advantage of SetFit is the ability to add new intents dynamically. To support a new command (e.g., "Open ChatGPT"), the system simply embeds the anchor phrase and adds the vector to the index. No model retraining is required, offering a significant feature advantage for user-customizable dictation apps.¹⁸

2.3 Classical Machine Learning Baselines

For the absolute lowest latency, classical NLP techniques utilizing TF-IDF or Bag-of-Words features fed into Logistic Regression or Support Vector Machines (SVMs) operate in the **sub-1ms** range.¹⁹

- **Limitations:** These models rely on lexical overlap and struggle with synonymy and context. They cannot distinguish "Book a flight" from "Book a hotel" if the distinguishing terms are not statistically dominant in the training set.
- **Role:** While insufficient as the primary classifier, they function excellently as a "heuristic guardrail" or "Level 0" filter for detecting exact-match system commands (e.g., "Stop listening") instantly.

2.4 Comparative Benchmark Projection

The following table projects the expected performance of these architectures on a standard mid-range Windows PC (e.g., Intel Core i5-12600K, no discrete GPU active).

Model Architecture	Parameters	Disk Size (INT8)	Est. CPU Latency	Est. Throughput	Accuracy (GLUE/SetFit)	Recommended Use
TinyBERT-4	14.5M	~15 MB	2 - 5 ms	>400 req/sec	~96.8% (BERT relative)	Primary Classifier
MobileBERT	25.3M	~26 MB	15 - 25 ms	~150 req/sec	77.7 (GLUE)	Secondary Alternative
DistilBERT	66M	~67 MB	30 - 50 ms	~80 req/sec	79.1 (GLUE)	Too slow for CPU fast-path

ModernBERT-Base	149M	~150 MB	60 - 90 ms	~40 req/sec	88.4 (GLUE)	Context-aware Editing
SetFit (MiniLM)	22M	~23 MB	10 - 15 ms	~300 req/sec	High (Few-Shot)	Dynamic /Custom Intents
Llama 3 8B (Q4)	8,000M	~4.5 GB	2000 - 8000 ms	<1 req/sec	State-of-the-Art	Fallback / Reasoning

Recommendation: The optimal architecture for WhisperLocal is **TinyBERT-4 (quantized to INT8)** running on the **ONNX Runtime CPU Execution Provider**. This configuration guarantees single-digit millisecond latency, minimal memory footprint, and sufficient capacity to distinguish between 50-100 distinct voice commands. For user-defined custom commands, a parallel **SetFit** index can be queried with negligible overhead.

3. Defining the Intent Space: A Taxonomy for Voice Dictation

Training a discriminative model requires a closed set of predefined labels. Unlike the open-ended generation of an LLM, the classifier must map every utterance to a specific bucket. The challenge in voice dictation is the inherent ambiguity between *content* (what the user wants to write) and *instruction* (what the user wants the system to do).

3.1 The "Command vs. Dictation" Ambiguity Problem

Consider the utterance: *"Select all"*

- **Interpretation A:** The user wants to execute the Select All command (Ctrl+A).
- **Interpretation B:** The user is dictating a sentence, e.g., "I need to select all the correct options."

If the classifier aggressively biases toward commands, users cannot dictate prose containing command words. If it biases toward dictation, commands are ignored. This is the central friction point in VUI design. Resolving this requires a taxonomy that explicitly separates these modalities and linguistic features (analyzed in Section 4) to disambiguate them.

3.2 Comprehensive Intent Taxonomy

Based on an analysis of mature dictation products like Dragon NaturallySpeaking²⁰, Windows Voice Access²³, and Talon Voice²⁴, the following taxonomy is proposed for WhisperLocal. Ideally, these should be hierarchical to allow for granular handling.

3.2.1 Formatting Intents (High Frequency)

These commands modify the text stream itself and are often interwoven with dictation. They require extremely low latency to maintain the user's flow.

- `format.new_line`: "New line", "Press enter", "Carriage return"
- `format.new_paragraph`: "New paragraph", "Next paragraph"
- `format.capitalization.cap`: "Cap that", "Capitalize", "Capitalize next"
- `format.capitalization.all_caps`: "All caps", "Uppercase", "Caps on"
- `format.capitalization.no_caps`: "Lowercase", "No caps", "Caps off"
- `format.spacing.no_space`: "No space", "Close up", "Space bar"
- `format.style.bold`: "Bold that", "Bold selection"
- `format.style.italic`: "Italicize that", "Italics"
- `format.style.underline`: "Underline that"

3.2.2 Punctuation Intents (Explicit Insertion)

While Whisper handles implicit punctuation well, users often dictate explicit punctuation to control structure.

- `punct.comma`: "Comma"
- `punct.period`: "Period", "Full stop", "Dot"
- `punct.question`: "Question mark"
- `punct.exclamation`: "Exclamation point", "Bang"
- `punct.quote.open`: "Open quote", "Quote"
- `punct.quote.close`: "Close quote", "End quote"
- `punct.bracket.open`: "Open bracket", "Left bracket"
- `punct.bracket.close`: "Close bracket", "Right bracket"

3.2.3 Editing Intents (Context-Aware)

These commands act upon the existing text buffer. The classifier needs to understand that these are imperative actions, not content.

- `edit.delete.prev_char`: "Backspace", "Delete character"
- `edit.delete.prev_word`: "Delete word", "Scratch word", "Delete last word"
- `edit.delete.prev_line`: "Delete line", "Scratch line"
- `edit.delete.range`: "Delete last [N] words", "Delete back to"
- `edit.delete.selection`: "Delete that", "Scratch that"
- `edit.replace`: "Change to", "Correct that"
- `edit.undo`: "Undo", "Undo that"
- `edit.redo`: "Redo", "Redo that"

3.2.4 Navigation & Selection Intents

- nav.move.start_line: "Go to start of line", "Home"
- nav.move.end_line: "Go to end of line", "End"
- nav.move.up: "Go up", "Line up"
- nav.move.down: "Go down", "Line down"
- nav.move.word_left: "Go left one word", "Previous word"
- nav.move.word_right: "Go right one word", "Next word"
- select.word: "Select", "Select current word"
- select.line: "Select line", "Select this line"
- select.all: "Select all"

3.2.5 System & Application Control

- sys.mic.stop: "Stop listening", "Go to sleep", "Pause mic"
- sys.mic.start: "Wake up", "Start listening"
- sys.app.switch: "Switch to [App Name]" (Requires disambiguation from dictation)
- sys.window.minimize: "Minimize window"

3.2.6 The Fallback Intent

- content.dictate: This class captures *everything* that does not fit into the above categories. In a typical session, 90-95% of utterances will fall into this class. Accurately classifying this is actually the most difficult task, as it requires the model to reject command-like phrases based on context.

3.3 Semantic Nuance Examples

To train a robust model, the data must reflect the nuances of how these intents manifest.

- **Imperative vs. Declarative:** "Delete this" (Imperative -> edit.delete.selection) vs. "I will delete this later" (Declarative -> content.dictate). The classifier must learn dependency patterns; imperative sentences often lack a subject and start with a verb.
- **Ambiguous Objects:** "Copy that" (Command) vs. "Can you copy that file?" (Request/Question). The presence of "Can you" or "Will you" shifts the probability toward dictation or a conversational intent (handled by LLM), whereas the isolated verb-object phrase strongly suggests a command.¹⁹

4. Hybrid Architecture: The Cascading Fallback Pattern

A sub-100ms classifier (TinyBERT) cannot handle every possible edge case. It lacks the world knowledge of Llama 3. Conversely, Llama 3 is too slow for "Delete word." The solution is a

Cascading Architecture that routes queries based on complexity and confidence.¹⁷

4.1 The Three-Stage Pipeline

The proposed architecture for WhisperLocal processes audio through three progressively more complex (and slower) stages.

Stage 1: The Heuristic Gate (Latency: < 0.1 ms)

Before any neural inference, the transcribed text passes through a deterministic filter.

- **Mechanism:** Aho-Corasick algorithm or FlashText.²⁷ These algorithms allow searching for thousands of keywords in a string in a single pass ($O(N)$ where N is the length of the string).
- **Task:** Detect "Wake Words" and "System Critical" commands (e.g., "Stop listening"). These must work 100% of the time with 0 latency.
- **Outcome:** If match found -> Execute immediately. Else -> Pass to Stage 2.

Stage 2: The Fast Neural Classifier (Latency: 2-10 ms)

This is the core of the system, powered by the **TinyBERT (INT8)** model running on ONNX Runtime CPU.

- **Input:** The raw text from Whisper.
- **Mechanism:** Forward pass through TinyBERT to obtain logits for the ~60 defined intents.
- **Confidence Check:** The system applies a **Softmax** function to the logits to get probabilities. It then calculates the **Entropy** of the distribution to measure uncertainty.²⁹
 - **High Confidence:** Probability(Top Class) > 0.85 AND Entropy < Threshold.
 - **Action:** If High Confidence, execute the intent (or type the text if content.dictate). If Low Confidence, pass to Stage 3.
- **Note:** The vast majority of dictation and formatting commands will be resolved here.

Stage 3: The LLM Fallback (Latency: 2-12 seconds)

If the user says something ambiguous like "Rewrite that to sound more professional," TinyBERT will likely have high entropy (confusion between content.dictate and edit.replace).

- **Action:** The system acknowledges the request (e.g., a "Thinking..." spinner in the UI) but does not block input.
- **Processing:** The text is sent to the local Llama 3 instance via Ollama.
- **Prompt:** A specific system prompt instructs Llama 3 to analyze the intent given the context of the previous sentences.
- **Outcome:** Llama 3 returns a structured JSON action, which the system executes asynchronously.

4.2 Mathematical Formulation of Confidence and Entropy

Relying solely on the highest softmax probability is dangerous; models can be "confidently wrong." **Entropy** provides a better measure of the model's internal confusion.

Given a set of probabilities $P = \{p_1, p_2, \dots, p_n\}$ for n intents, the Shannon Entropy $H(P)$ is:

$$H(P) = - \sum_{i=1}^n p_i \log_2(p_i)$$

- **Low Entropy:** The distribution is peaked; the model is certain. (e.g., $p_{delete} = 0.98$, others ≈ 0).
- **High Entropy:** The distribution is flat; the model is guessing. (e.g., $p_{delete} = 0.4$, $p_{dictate} = 0.35$, $p_{replace} = 0.25$).

Routing Logic:

Python

```
def route_intent(logits):
    probs = softmax(logits)
    entropy = -sum(probs * log2(probs))
    max_conf = max(probs)

    # Thresholds tuned via validation set
    if entropy < 1.0 and max_conf > 0.8:
        return "FAST_EXECUTE"
    else:
        return "LLM_FALLBACK"
```

4.3 Integrating Acoustic Features (Pause Detection)

Text alone is often insufficient. "Period" is a punctuation mark; "Period" is also a length of time. The distinction in speech is often **silence**. A command is typically isolated by pauses.

- **Implementation:** Whisper supports word-level timestamps. By calculating the inter-word silence, we can create a powerful feature.³¹
 - Pre-Pause = Word_Start - Previous_Word_End

- $\text{Post-Pause} = \text{Current_Time} - \text{Word_End}$
- **Heuristic:** If $\text{Pre-Pause} > 400\text{ms}$ AND $\text{Post-Pause} > 400\text{ms}$, boost the probability of Command intents. If the word is spoken rapidly within a stream of words, force content.dictate. This logic effectively acts as a dynamic prior for the Bayesian classification.

5. Synthetic Data Generation Strategy (The "Teacher-Student" Loop)

The success of the TinyBERT student model relies entirely on the quality of its training data. Since no public dataset exists for "Whisper-transcribed Windows voice commands," we must generate one. This process, known as **Data Synthesis** or **Knowledge Distillation via Generation**, utilizes the capabilities of the "Teacher" LLM (Llama 3) to create a training corpus for the "Student" (TinyBERT).⁸

5.1 The Master Prompt Strategy

Simple prompts like "Generate 100 variations of 'delete word'" lead to **mode collapse**—the LLM outputs repetitive, low-diversity patterns. To train a robust classifier, we need **Attribute-Based Prompting**³⁵ to force linguistic diversity.

Teacher: Llama 3 8B (Local)

Objective: Generate 50,000+ labeled examples across the 60+ intent categories.

5.1.1 Prompt Template for Command Generation

We must instruct the LLM to vary the **register** (formal vs. casual), **phrasing**, and **noise** (fillers).

System Prompt:

"You are an expert computational linguist creating a dataset for a voice dictation system. Your goal is to generate realistic user utterances for specific intents."

User Prompt:

"Generate 50 unique utterance variations for the intent: edit.delete_last_sentence."

Constraints & Diversity Parameters:

1. **Direct Commands:** (e.g., 'Delete the last sentence.')
2. **Frustrated/Casual:** (e.g., 'No, scratch that whole sentence.')
3. **With Fillers:** (e.g., 'Um, actually, remove the last sentence.')

4. **Implicit/Contextual:** (e.g., 'That was wrong, take it back.')
5. **False Starts:** (e.g., 'Delete... actually delete the last sentence.')

Output Format:

Return ONLY a valid JSON array of objects: [{"text": "...", "intent": "edit.delete_last_sentence"},...]"

5.2 The "Hard Negative" Challenge (The Dictation Class)

The most critical part of the dataset is the content.dictate class. The model must learn that "delete the last word" is a command, but "I decided to delete the last word of the chapter" is dictation. We need to generate **Hard Negatives**—sentences that contain command keywords but are semantically declarative.¹⁷

Prompt for Hard Negatives:

"Generate 50 declarative sentences that naturally contain the words: 'delete', 'select', 'copy', 'paste', 'enter', 'new line'.

Crucial: These must be sentences a user would dictate, NOT commands to the system.

- *Bad Example:* 'Select all text.' (This is a command).
- *Good Example:* 'He told me to select all the red apples from the bin.' (This is dictation).

Label all these as content.dictate."

5.3 Distillation Pipeline Implementation

1. **Schema Definition:** Define the 60+ intents in a YAML file.
2. **Batch Generation:** Write a Python script to iterate through the schema, sending prompts to the local Ollama API. Generate ~500 examples per command intent (30,000 total) and ~20,000 examples of hard-negative dictation.
3. **Filtration:** Use a keyword filter to ensure the generated "hard negatives" actually contain the target keywords (otherwise they are too easy).
4. **Consistency Check:** (Optional) Use a smaller model or a second pass with Llama 3 to verify that the labels make sense.
5. **Data Split:** 80% Train, 10% Validation, 10% Test.

6. Implementation Strategy: Windows, ONNX, and Optimization

6.1 The ONNX Runtime (ORT) Environment

To deploy TinyBERT on Windows efficiently, we utilize Microsoft.ML.OnnxRuntime.

- **Execution Provider:** CPUExecutionProvider is recommended. While DmlExecutionProvider (DirectML) allows GPU usage, the memory transfer overhead for a model as small as TinyBERT often negates the compute speedup. Furthermore, keeping the intent classifier on the CPU leaves the GPU VRAM entirely free for the heavier Whisper transcription model.
- **Quantization:** The model should be quantized to **INT8** using the OnnxRuntime.Quantization tools. This converts 32-bit floating-point weights to 8-bit integers.
 - *Benefit:* Reduces model size from ~60MB to ~15MB.
 - *Benefit:* Enables the use of AVX-512 VNNI instructions on modern CPUs, potentially doubling inference throughput.⁵

6.2 C# Integration Example (Conceptual)

For a Windows native app (WPF/WinUI 3):

C#

```
using Microsoft.ML.OnnxRuntime;
using Microsoft.ML.OnnxRuntime.Tensors;

public class IntentClassifier
{
    private InferenceSession _session;

    public IntentClassifier(string modelPath)
    {
        var options = new SessionOptions();
        options.GraphOptimizationLevel = GraphOptimizationLevel.ORT_ENABLE_ALL;
        options.ExecutionMode = ExecutionMode.ORT_SEQUENTIAL;
        // Use CPU provider for lowest latency on small models
        options.AppendExecutionProvider_CPU(1);
        _session = new InferenceSession(modelPath, options);
    }

    public IntentResult Classify(string text)
    {

```

```
// Tokenization logic (omitted for brevity) needed here to convert text to input_ids
var inputs = new List<NamedOnnxValue> { /*... */ };

using var results = _session.Run(inputs);
var logits = results.First().AsTensor<float>();

// Apply Softmax and Calculate Entropy
// Return Intent and Confidence
}
```

6.3 Threading Model

Since Whisper and the Intent Classifier must run in near real-time, threading is critical.

- **Whisper Thread:** Dedicated high-priority thread for audio processing.
- **Classification Thread:** The Intent Classifier should run on a separate thread pool. Since inference is <5ms, it will rarely block.
- **UI Thread:** Only updated upon final decision. Optimistic UI (showing the text *before* the intent is fully confirmed) can make the system feel faster, with a rapid rollback/correction if a command is detected.

7. Benchmarking & Expectations

Based on the synthesis of research data, the expected performance improvements of this hybrid architecture over the current pure-LLM approach are substantial.

Metric	Current (Llama 3 8B)	Hybrid (TinyBERT INT8 + Cascade)	Improvement Factor
P99 Latency	2,000 - 12,000 ms	5 - 15 ms	~400x - 800x
Memory Footprint	4 - 8 GB VRAM	~20 MB RAM	~200x
Throughput	0.5 - 2 req/sec	>300 req/sec	~150x
Hardware Reqs	High-end GPU rec.	Any x64 CPU	Universal Compatibility
Accuracy	High (Contextual)	High (Domain)	Comparable (in

		Specific)	domain)
--	--	-----------	---------

8. Conclusion

The "12-second latency problem" in WhisperLocal is not a limitation of local hardware, but a limitation of architecture. By forcing a generative giant (Llama 3) to perform a discriminatory chore, resources are squandered.

The path forward is the **Distilled Hybrid Cascade**:

1. **Train** a domain-specific TinyBERT classifier using Llama 3 to generate the training data.
2. **Deploy** this model using ONNX Runtime (CPU/INT8) for sub-100ms inference.
3. **Route** ambiguous queries to the LLM only when necessary (high entropy).
4. **Refine** the logic using acoustic pauses from Whisper timestamps.

This architecture shifts the user experience from "frustratingly slow" to "perceptually instant," unlocking the true potential of offline voice interfaces on Windows.

Works cited

1. Tiny Language Models for Automation and Control: Overview, Potential Applications, and Future Research Directions - PMC, accessed February 10, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11902656/>
2. Running Small Language Models (SLMs) on CPUs: A Practical Guide | by Devi - Towards AI, accessed February 10, 2026, <https://pub.towardsai.net/running-small-language-models-slms-on-cpus-a-practical-guide-aa976d7ecd41>
3. Comparative Analysis of Compact Language Models for Low-Resource NLP: A Study on DistilBERT, TinyBERT, and MobileBERT - Zenodo, accessed February 10, 2026, <https://zenodo.org/records/15907007>
4. Microsoft open sources breakthrough optimizations for transformer inference on GPU and CPU, accessed February 10, 2026, <https://opensource.microsoft.com/blog/2020/01/21/microsoft-onnx-open-source-optimizations-transformer-inference-gpu-cpu>
5. ONNX Runtime Performance Tuning - GitHub Pages, accessed February 10, 2026, <https://iot-robotics.github.io/ONNXRuntime/docs/performance/tune-performance.html>
6. Get started with DirectML | Microsoft Learn, accessed February 10, 2026, <https://learn.microsoft.com/en-us/windows/ai/directml/dml-get-started>
7. Deploy High-Performance AI Models in Windows Applications on NVIDIA RTX AI PCs, accessed February 10, 2026, <https://developer.nvidia.com/blog/deploy-ai-models-faster-with-windows-ml-on-rtx-pcs/>
8. Knowledge distillation and dataset distillation of large language models: emerging

- trends, challenges, and future directions - PMC, accessed February 10, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12634706/>
9. Knowledge distillation: Teaching LLM's with synthetic data | ML_NEWS3 - Wandb, accessed February 10, 2026,
https://wandb.ai/byyoung3/ML_NEWS3/reports/Knowledge-distillation-Teaching-LLM-s-with-synthetic-data--Vmlldzo5MTMyMzA2
 10. DistilBERT, ALBERT, and Beyond: Comparing Top Small Language Models - C# Corner, accessed February 10, 2026,
<https://www.c-sharpcorner.com/article/distilbert-albert-and-beyond-comparing-top-small-language-models/>
 11. Benchmark ONNX runtime performance with onnxruntime_perf_test - Arm Learning Paths, accessed February 10, 2026,
<https://learn.arm.com/learning-paths/servers-and-cloud-computing/onnx-on-azure/benchmarking/>
 12. MobileBERT - Hugging Face, accessed February 10, 2026,
https://huggingface.co/docs/transformers/en/model_doc/mobilebert
 13. SetFit ModernBERT for text classification with few-shot training! | by Moshe Wasserblat, accessed February 10, 2026,
<https://moshewasserblat.medium.com/new-results-on-setfit-modernbert-for-text-classification-with-few-shot-training-53c154df7c0e>
 14. Finally, a Replacement for BERT: Introducing ModernBERT - Hugging Face, accessed February 10, 2026, <https://huggingface.co/blog/modernbert>
 15. ModernBERT vs BERT: Key Differences and Advantages - EnkiAI, accessed February 10, 2026,
<https://enki.ai/modernbert-vs-bert-advantages-and-differences>
 16. TrueFoundry Accelerator Series: Building Enterprise-Grade Intent Classification with SetFit, accessed February 10, 2026,
<https://www.truefoundry.com/blog/truefoundry-accelerator-series-building-enterprise-grade-intent-classification-with-setfit>
 17. Intent Detection in the Age of LLMs - arXiv, accessed February 10, 2026,
<https://arxiv.org/html/2410.01627v1>
 18. Intent Classification in <1ms: How We Built a Lightning-Fast Classifier with Embeddings | by Durgesh Rathod | Medium, accessed February 10, 2026,
<https://medium.com/@durgeshrathod.777/intent-classification-in-1ms-how-we-built-a-lightning-fast-classifier-with-embeddings-db76bfb6d964>
 19. Intent Classification: Techniques for NLP Models - Data Annotation Company, accessed February 10, 2026,
<https://labelyourdata.com/articles/machine-learning/intent-classification>
 20. Recognition modes, accessed February 10, 2026,
<https://www.nuance.com/products/help/dragon/dragon-for-mac6/enx/Content/Introduction/RecognitionModes.html>
 21. Dragon Professional Individual Command Cheat Sheet - Nuance, accessed February 10, 2026,
https://www.nuance.com/asset/en_us/collateral/dragon/command-cheat-sheet/ct-dragon-professional-individual-en-us.pdf

22. Dragon NaturallySpeaking Cheat Sheet - Nuance, accessed February 10, 2026, https://www.nuance.com/asset/en_us/collateral/dragon/command-cheat-sheet/ct-dragon-naturally-speaking-en-us.pdf
23. Voice access command list - Microsoft Support, accessed February 10, 2026, <https://support.microsoft.com/en-us/topic/voice-access-command-list-dac0f091-87ce-454d-8d57-bef38d3d8563>
24. Basic Usage - Talon Community Wiki, accessed February 10, 2026, https://talon.wiki/Basic%20Usage/basic_usage/
25. 4 Types of Sentences: Understanding Function and Structure - EssayPro, accessed February 10, 2026, <https://essaypro.com/blog/types-of-sentences>
26. Towards Efficient Multi-LLM Inference: Characterization and Analysis of LLM Routing and Hierarchical Techniques - arXiv, accessed February 10, 2026, <https://arxiv.org/html/2506.06579v1>
27. Real Solution for Keyword Matching and Replacing in python — FlashText | by punna111, accessed February 10, 2026, <https://medium.com/@punna111/real-solution-for-keyword-matching-and-replacing-in-python-flashtext-178cfdd060e2>
28. Why You Should Use FlashText Instead of RegEx for Data Analysis - Alibaba Cloud, accessed February 10, 2026, <https://www.alibabacloud.com/blog/593727>
29. Entropy, Cross-Entropy, and How LLMs Learn What Comes Next | by Stas Spivak | Medium, accessed February 10, 2026, <https://medium.com/@spybacks/entropy-cross-entropy-and-how-llms-learn-what-comes-next-80e2334e4708>
30. Entropy in machine learning — applications, examples, alternatives - Nebius, accessed February 10, 2026, <https://nebius.com/blog/posts/entropy-in-machine-learning>
31. Preserving audio pauses? · openai whisper · Discussion #2073 ..., accessed February 10, 2026, <https://github.com/openai/whisper/discussions/2073>
32. linto-ai/whisper-timestamped: Multilingual Automatic Speech Recognition with word-level timestamps and confidence - GitHub, accessed February 10, 2026, <https://github.com/linto-ai/whisper-timestamped>
33. Scaling Low-Resource MT via Synthetic Data Generation with LLMs - ACL Anthology, accessed February 10, 2026, <https://aclanthology.org/2025.emnlp-main.1408.pdf>
34. Using large language models (LLMs) to synthesize training data - Amazon Science, accessed February 10, 2026, <https://www.amazon.science/blog/using-large-language-models-llms-to-synthesize-training-data>
35. A Novel Llama 3-Based Prompt Engineering Platform for Textual Data Generation and Labeling - MDPI, accessed February 10, 2026, <https://www.mdpi.com/2079-9292/14/14/2800>
36. Common Commands in Speech Recognition, accessed February 10, 2026, <https://www.mvcc.edu/accessibility-resources/pdf/CommonCommandsInSpeechRecognition.pdf>
37. How to compute LLM embeddings 3X faster with model quantization - Medium,

accessed February 10, 2026,

<https://medium.com/nixiesssearch/how-to-compute-llm-embeddings-3x-faster-with-model-quantization-25523d9b4ce5>